The MoE Memory Wall

Predicting and Prefetching Experts for Faster LLM Inference.
(https://github.com/breaking-byts/MoE-Memory-Wall-CA-Project)

Introduction

Usually the large language models are served in two phases.

- The Prefill Phase which takes in the whole prompt at once and is majorly limited by compute.
- The Decode Phase takes place after the prefill phase and it produces one token at a time, and for every new token generation the machine has to fetch the model weights from the memory again while doing a small amount of arithematic.

So essentially decode is a memory bandwidth problem, the time required for one step is roughly the bytes of weights divided by the memory bandwidth. Mixture-of-Experts (MoE) make this a lot harder.Instead of a single large feed-forward network per layer, an MoE layer has many "experts" and a router that chooses the top-2 experts for each token.The router decides which expert a token needs only when the token reaches that layer, and the decision depends on the token's hidden state, and since the system cannot know which two will be picked in advance, all of the experts must be stored and kept ready. That is the problem this project is about: the MoE memory wall.

For this project we had to:

1. Reproduce the Duplex configuration in LLMSimulator and show a working simulation.
2. Disable multi-GPU sharding and Logic-PIM, put all expert weights in off-chip DRAM, and measure expert loading time versus expert execution time.
3. Understand the MoE execution flow and explain why expert loading is the bottleneck.
4. Check whether expert accesses are predictable, using plots and statistics, as the first step towards a prefetcher.

Background

1. Why decode is bandwidth bound ?

For a weight matrix of k x n elements, one decode step with m tokens reads 2kn bytes from the memory and perform 2mkn floating-point operations. The arithematic intensity (operations per byte) is hence just m, that is the no. of tokens. A machine's efficiency is given by the roofline model : a kernel's time is :

$$t = \max\left(\frac{\text{FLOPs}}{\text{peak FLOPs}}, \frac{\text{bytes}}{\text{bandwidth}}\right)$$

and below the ridge point (Peak Flops divided by Bandwidth) a kernel is memory bound. For FP16 H100 that ridge point is 989.4 TFLOPS/ 3.352 TB/s which roughly equals 295 Op/B. In

MoE decode an expert sees only 1-3 tokens, which is a about a 100 times below the ridge, so the expert projections spend almost all their time reading the weights.

2. Mixtral-8x7B and why the experts are the problem ?

Mixtral has 32 layers and hidden size 4096, grouped-query attention,and 8 experts per layer routed top-2. Each expert contains a feed-forward network with three matrices, so :

$$\text{one expert} = 3 \times 4096 \times 14336 = 176.16\text{M parameters} = 352.3\ \text{MB in FP16}.$$

The experts therefore makeup 45.10 B of the model's total 46.7 billion parameters, and the weights alone occupy 86.99 GiB. Each of the 8 experts per layer is used only by a couple of tokens per step but at a batch size of 8 there is little reuse of expert weights from token to token, yet a mean of 7.28 out of 8 experts is active per layer, so nearly a full 2.82 GB of weights per layer is streamed per step.

3. The memory hierarchy

The GPU is H100 HBM3 : 80 GiB capacity, 3.352 TB/s bandwidth. When the experts are kept off chip they live in the CPU DRAM behind an NVLink-C2C 5.0 link of 900 GB/s - 3.7x slower to reach. (A host chip behind the PCIe would still be slower so the 68.13 ms loading time is optimistic.)

4. Duplex

Duplex attaches a Logic-PIM die to the same HBM stacks as the GPU. The GPU handles the compute intensive work while Logic-PIM serves the bandwidth-heavy small-batch decode experts, and since both these chips sit on the same HBM so no extra data movement is introduced.In LLMSimulator this is GPU+LOGIC processor type.

5. The Simulators

We have made use of two simulators as prescribed in the project requirements:

- LLMSimulator lists all the operations in the model (QKV projection, attention, etc) in the order they are meant to run.And for each of the operations it calculates the time required for the arithmetic and reading the data from the memory and reports the higher of the two times since that would be the bottleneck for the particular task. It also calculates the energy required for each operation uses and it does it all based off the yaml config file which describes the hardware and workload in a text file.The routing is synthetic which means that the simulator doesn't really run a real model. It makes up choices. Uniform by default meaning each expert has a equal likelihood of being chosen or Zipf-skewed which makes likelihood of a few experts being chosen higher than the other experts.

- SSD-offloading is a modified version of LLMSimulator that adds the ability to the keep the model weight in the CPU memory instead of on the GPU.The `expert_ffn` measures one of the two different things based off the `collects_gpu_compute` flag:
  - **off:** the time to copy the experts from CPU to GPU memory (the loading cost).
  - **on:** the time for the GPU to execute the experts (the execution cost).

Experimental Setup

All runs use the Mixtral model in FP16 on an H100-class device. The five configs we wrote for this report:

Config	Simulator	Setup	Used for
`mixtral_gpu.yaml`	LLMSim	4 GPUs, DP4, batch 32, 1024/128 tokens	D1 baseline
`mixtral_duplex.yaml`	LLMSim	4 GPUs, `GPU+LOGIC`, TP4	D1 Duplex
`mixtral_single_gpu.yaml`	LLMSim	1 GPU, batch 8, op log on	D2 OOM, D3 flow
`ssd_cfgs/cpu_copy.yaml`	SSD-off	experts in CPU DRAM, copy mode	D2 loading
`ssd_cfgs/cpu.yaml`	SSD-off	experts in CPU DRAM, compute mode	D2 execution

The two D1 configs differ only in `processor_type` (GPU vs GPU+LOGIC) , the parallel degrees (1/1 = data parallel vs 4/4 = tensor parallel), and parallel_execution (off vs on) and the two D2 configs differ only in `collects_gpu_compute` flag. Both simulators were built in an Ubuntu 22.04 container (with g++-11, C++17; git is needed at configure time). Each LLMSimulator run writes a log with the memory footprint and a total time, plus one row per iteration while each SSD-Offloading run writes a CSV with per-operation latencies and energies.Three short Python scripts(`analysis_d2.py, analysis_d3.py, analysis_d4.py`) parse these outputs and re-derive the numbers.There exists a quirk in the `LLMSimulator/src/hardware/cluster.cpp` where n iterations write n-1 rows since the last iteration's computation is never exported.We always use the row 0.

Results

1. Deliverable 1 : reproducing Duplex

Same workload on 4 H100s, once as a plain GPU System and once as a Duplex. Both are decode only slices : batch 32 , 1024 inputs / 128 output tokens and FP16 precision.

Metric	GPU	Duplex	Change
Total time, 3 iterations (log)	23.943 ms	7.726 ms	**3.10×**
Latency per iteration (CSV)	7.980 ms	2.574 ms	3.10×
Expert FFN (`expert_ffn`)	6765 µs	1721 µs	3.93×
Attention (`atten_gen`)	337 µs	81 µs	4.14×
QKV / output projections	482 / 322 µs	124 / 83 µs	~3.9×
Communication	69 µs	545 µs	TP all-reduce
Energy per iteration	3.228 J	3.160 J	−2.1%

Duplex is 3.10x faster at decode and most of the win comes from expert_ffn, which is approximately 85% of the time taken per step. The Logic-PIM serves the expert directly from the shared HBM.The projection weights shrink by ~4× from tensor-parallel sharding, while \atten_gen` (337 to 81 µs) is and the new 545 us all-reduce communication is the price of that sharding.The benefit that we got was in latency. And to identify which of the three changes was responsible, we ran two extra variants. TP4 sharding alone(without Logic-PIM) barely changes anything as the experts are really bandwidth bound and sharding doesn't change that; while turning the Logic-PIM on gives a real jump (23.641 to 8.258 ms,2.86x), and `parallel_execution` on adds another 7%. So we can say almost all of the speed up can be atributted to the Logic-PIM. Here we are studying the slice that is decode-only, which is the best case for the Logic-PIM. And the printed footprints - 26.07 GiB for the GPU run and 23.96 GiB for Duplex - differ because under data parallelism the experts are split 4 ways while attention is replicated , whereas under TP4 all shardable weights are split and the embeddings stay replicated.

2. Deliverable 2: single GPU with experts in off-chip DRAM

The model doesn't fit will all of its weight in one H100, the log reads ACT: 0.95 GiB, Weight: 86.99 GiB, Cache: 0.53 GiB, Total: 88.48 GiB which is more than the 80 GiB of HBM due to which the CSV even raises an OOM flag). And since the weights here alone exceed the HBM , no amount of batch sizing would fix this. To make this work the experts go to the off-chip CPU DRAM. We then ran the SSD-offloading framework with experts in CPU memory (batch 4, one decode step over the 32 layers). The same `expert_ffn` number can mean two different things depending upon the flag :

Config	`collects_gpu_compute`	`expert_ffn` means	Result
`ssd_cfgs/cpu_copy.yaml`	off	copy routed experts CPU→HBM	**68.13 ms**

Config	`collects_gpu_compute`	`expert_ffn` means	Result
`ssd_cfgs/cpu.yaml`	on	GPU execution of the experts	**25.02 ms**

Loading the experts is costs 2.72× as much as executing them. The numbers also match up with the theory closely. The run moves 61.31 GB of expert bytes which would be 174 expert loads of 352.3 MB that is 5.44 distinct experts per layer, which is what would be the expected behaviour for 4 tokens drawing 2 of 8 experts each :

$$8\left(1-\left(\frac{6}{8}\right)^4\right)=5.47.$$

$$\text{load}=\frac{\text{bytes}}{900\ \text{GB/s}},\qquad \text{exec}=\frac{\text{bytes}}{3352-900\ \text{GB/s}},\qquad \frac{\text{load}}{\text{exec}}=\frac{2452}{900}=2.72\times$$

The striking part is that the execution itself is memory bound. Its arithmetic is only 0.091 ms which is 275 x less than the measured 25.02 ms, which is pure HBM Bandwidth. So the load to execution ratio is not a plain link versus HBM (which would be 3352/900 = 3.72×) . While the connection copies the experts into HBM at 900 GB/s, that copy traffic itself occupies 900 GB/s of HBM's bandwidth, leaving 2452 GB/s for execution. In terms of energy consumption the CPU→HBM copy costs 3.19× the HBM read (6.57 J vs 2.06 J), which follows from the framework's per-bit constants (13.4 vs 4.2 pJ/bit). There is something to keep in mind while reading these numbers though which is that the framework has no expert caching which means that it reloads every routed expert every step , which would be the worst case scenario. And in copy mode the offload flag applies to every linear layer, not just the experts, hence it also charges a copy for attention weights though they never leave the GPU and hence only the `expert_ffn` column is meaningful.

3. Deliverable 3: where the time goes inside a decode step

We re-ran the single-GPU config with the operation log on (this changes no results).Layer 0 of the parsed op tree:

```
MoE_decoder_0         872.5us
  attention            30.5us   <- 3.5% of the layer
    attn_qkv_proj      15.1us   [8x4096] x [4096x6144]
    self_attention      5.4us   (flash attention)
    attn_o_proj        10.0us
  expertFFN           841.8us   <- 96.5% of the layer
    gate_fn           0.039us   4096 -> 8 router scores
    moe_route             0us   tokens/expert = [3, 2, 2, 1, 2, 2, 3, 1]
    expert_FFN_0      105.3us
      gate_proj        35.1us   [3x4096] x [4096x14336]
```

```
      up_proj          35.1us   [3x4096] x [4096x14336]
      down_proj        35.1us   [3x14336] x [14336x4096]
    expert_FFN_1..7    ~105us each, back-to-back
```

The simulator reports a singular fused time per operation, max(compute,memory) so we can verify which of the two dominates by replaying the arithmetic for first active `gate_proj` ([3×4096]×[4096×14336]):

$$\begin{aligned}\text{weight read} &= 4096 \times 14336 \times 2\,\text{B} = 117.4\,\text{MB},\\ t_{\text{mem}} &= 117.4\,\text{MB}/3.352\,\text{TB/s} = 35.036\ \mu\text{s} \quad (\text{measured } 35.069\ \mu\text{s}),\\ t_{\text{cmp}} &= 352.3\,\text{MFLOP}/989.4\,\text{TFLOPS} = 0.356\ \mu\text{s}.\end{aligned}$$

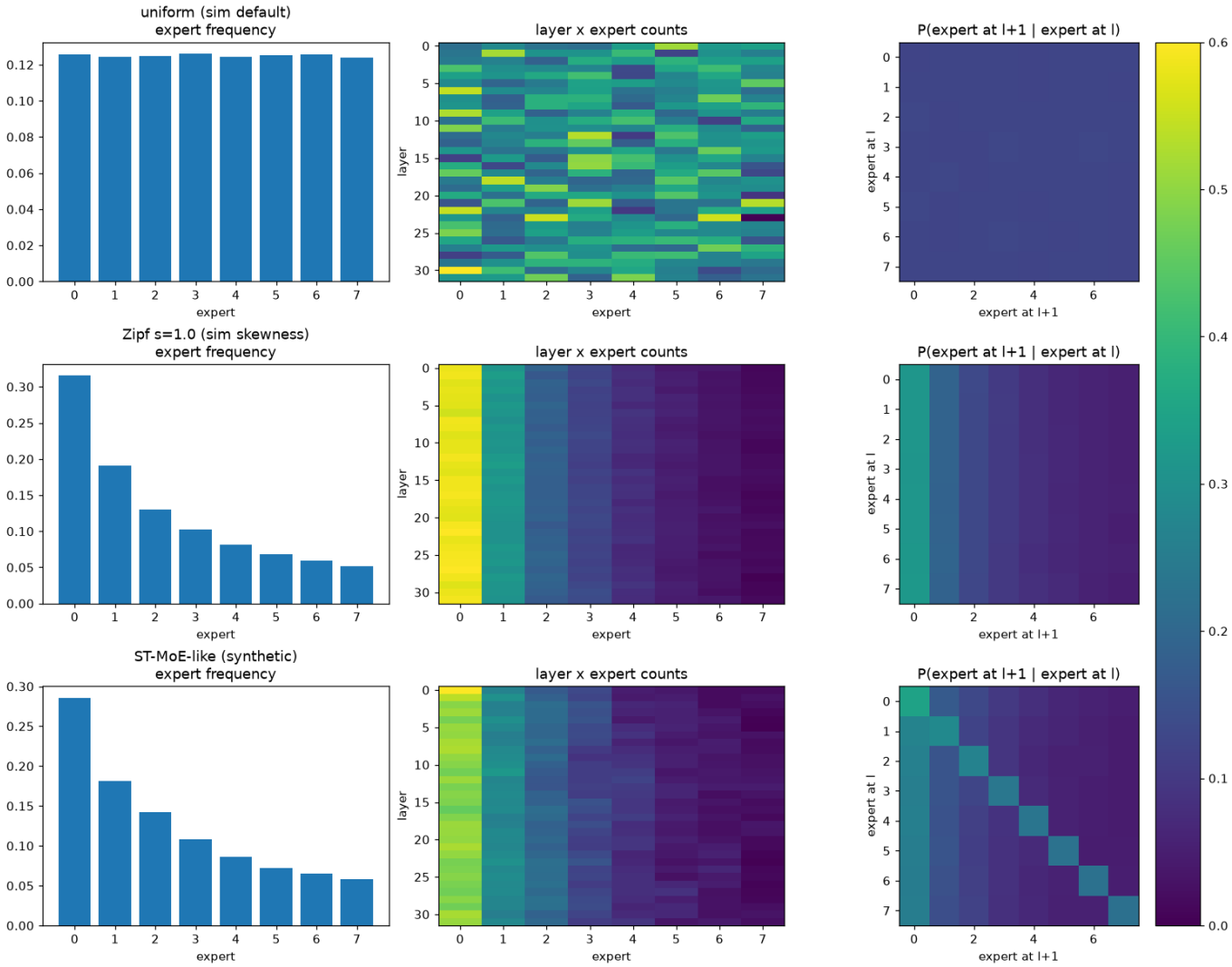
As visible here reading the weights takes 98x longer as compared to the arithmetic, the factor is simply the ridge point divided by the number of tokens an op serves: 295/3 ≈98×here. The expert up proj and down proj have the same 117.4 MB weight shape, so the same holds for them; the attention projections serve all 8 tokens, so their factor is only 295/8 ≈ 37× memory bound too, but smaller, which is why the expert FFN block dominates. Over all 32 layers, expert FFNs are 96.2% of decoder time and a mean of 7.28 of 8 experts is active per layer. Loading is the bottleneck essentially because the bytes are many and tokens are few.

4. Deliverable 4: can expert accesses be predicted?

LLMSimulator ships no real routing traces, so analysis `d4.py` generates three scenarios itself: the simulator's uniform routing, its Zipf skewed routing (s= 1.0), and one synthetic ST MoE like trace with stickiness (a token repeats the previous token's experts with probability 0.7, and otherwise each expert from the previous layer is kept with probability 0.6). The third one is not a measurement of real Mixtral routing — its correlation is set by those two knobs — but it lets us test the analysis method.

D4: routing statistics from `analysis_d4.py`. The uniform row's coverage is the script's re-implementation of default routing; the simulator's own batch-8 run gives 7.28 (D3).

Routing	Top expert share	Layer overlap	Token overlap	Experts/step
uniform (modelled default)	0.126	0.251	0.249	7.24
Zipf $s = 1.0$ (skewness mode)	0.315	0.362	0.361	6.45
ST-MoE-like (synthetic)	0.286	0.556	0.825	6.56



The overlap expected by pure chance is 2/8 = 0.25; The figure below shows the same statistics as plots. Three things stand out:

- The simulator's routing has **no** correlation — both overlaps sit on the 0.25 null and the transition matrix is flat. Any predictor we build must be evaluated on traces with real structure.
- Zipf skew only concentrates traffic on hot experts (one expert takes 31.5% of accesses, overlap rises to 0.36). The transition matrix shows vertical stripes — *popularity*, not *predictability*.
- Only the sticky trace gives a diagonal transition matrix and overlaps well above the null (0.556 layer-to-layer, 0.825 token-to-token) — that diagonal is what prediction exploits.

The consequence for our prefetcher plans: correlation can tell us *which* experts to fetch early, but it barely reduces *how many* we must fetch — coverage stays between 6.45 and 7.24 of 8 experts in every case.

Discussion

1. What we learned

All of the four results tell a singular story. The process of decoding is bound by the bandwidth; expert weights are humongous (352.3 MB each) and touch only 1–3 tokens, so 96.2% of decoding time is spent reading weights of the experts (D3). Moving those experts off the chip puts them behind a 900 GB/s connection, and because the copy and the GPU's execution utilise the same HBM, execution is left with only 2452 GB/s — which is what makes loading cost 2.72× of what execution costs (D2). A PIM die that shares the HBM removes that extra hop for the bandwidth-bound experts, and that is where the 3.10× comes from (D1). Whether prefetching can hide the rest depends on routing correlation (D4), which the stock simulator does not have.

2. Limitations

- Both simulators are analytical meaning that there is no DRAM timing, no contention, no queueing. The numbers are idealized bounds, rather than measurements from real hardware.
- The D2 framework has no expert caching, so 68.13 ms is the pessimistic on-demand cost.
- The D1 speed-up is a decode-only slice which would be the best case for Logic-PIM; a mixed prefill+decode workload would show a lesser of a gain.
- D4's correlated trace is synthetic. It validates our analysis, not real Mixtral routing — real traces are needed before claiming any prediction accuracy.
- The 900 GB/s off-chip link is a relatively fast interconnect; a slower host would make loading even more dominant.

3. Reproducibility

Everything was regenerated from the files in the project folder: both simulators were rebuilt from scratch in a clean container, D1 and single-GPU logs came out byte-identical, the D2 columns reproduced exactly (one unused energy column drifted by 5×10^{-8}, a floating-point artefact), and all three analysis scripts exit 0.

Bibliography

[1] S. Yun et al., “Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching,” arXiv:2409.01141 [cs.AR], 2024.

[2] Y. Zhao, R. Bunescu, A. Louri, A. Karanth, and K. Wang, “A spatio-temporal expert prefetching framework for efficient MoE-based LLM inference,” arXiv:2606.15453 [cs.AR], 2026.

[3] A. Kanani, L. Badawi, and U. Y. Ogras, “APEX: Adaptive expert prefetching for memory-efficient edge MoE inference,” arXiv:2608.11688, 2026.

[4] A. Q. Jiang et al., “Mixtral of experts,” arXiv:2401.04088, 2024.

[5] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 60–67, 2009.

[6] NVIDIA Corporation, “NVIDIA H100 Tensor Core GPU architecture,” whitepaper, 2022.

[7] scale-snu, “LLMSimulator,” https://github.com/scale-snu/LLMSimulator.

[8] scale-snu, “SSD-offloading,” https://github.com/scale-snu/SSD-offloading.

Appendix A

Run Commands

```
# build (Ubuntu 22.04 container with g++-11, cmake, make, git)
cd LLMSimulator && mkdir -p build data && cd build && \
  cmake -DCMAKE_CXX_COMPILER=g++-11 -DCMAKE_C_COMPILER=gcc-11 \
    -DCMAKE_CXX_STANDARD=17 .. && \
  make -j4
cd ../../SSD-offloading/LLMSimulator && mkdir -p build && cd build && \
  cmake -DCMAKE_CXX_COMPILER=g++-11 -DCMAKE_C_COMPILER=gcc-11 \
    -DCMAKE_CXX_STANDARD=17 .. && \
  make -j4 run

# D1 + D2 OOM + D3 op tree
cd LLMSimulator/build && \
  ./run ../../mixtral_gpu.yaml        > ../../run_mixtral_gpu.log && \
  ./run ../../mixtral_duplex.yaml     > ../../run_mixtral_duplex.log && \
  ./run ../../mixtral_single_gpu.yaml > ../../run_mixtral_single.log

# D2 load vs exec (the SSD framework reads ./config.yaml from the CWD)
cd SSD-offloading/LLMSimulator/build && \
  cp ../../../ssd_cfgs/cpu.yaml config.yaml      && ./run >
../../../ssd_results/ssd_cpu.log && \
  cp ../../../ssd_cfgs/cpu_copy.yaml config.yaml && ./run >
../../../ssd_results/ssd_cpu_copy.log && \
  cp tmp/*.csv ../../../ssd_results/

# analysis
python3 analysis_d2.py
python3 analysis_d3.py
```

```
python3 analysis_d4.py     # needs numpy + matplotlib, writes
d4_correlation.png
```